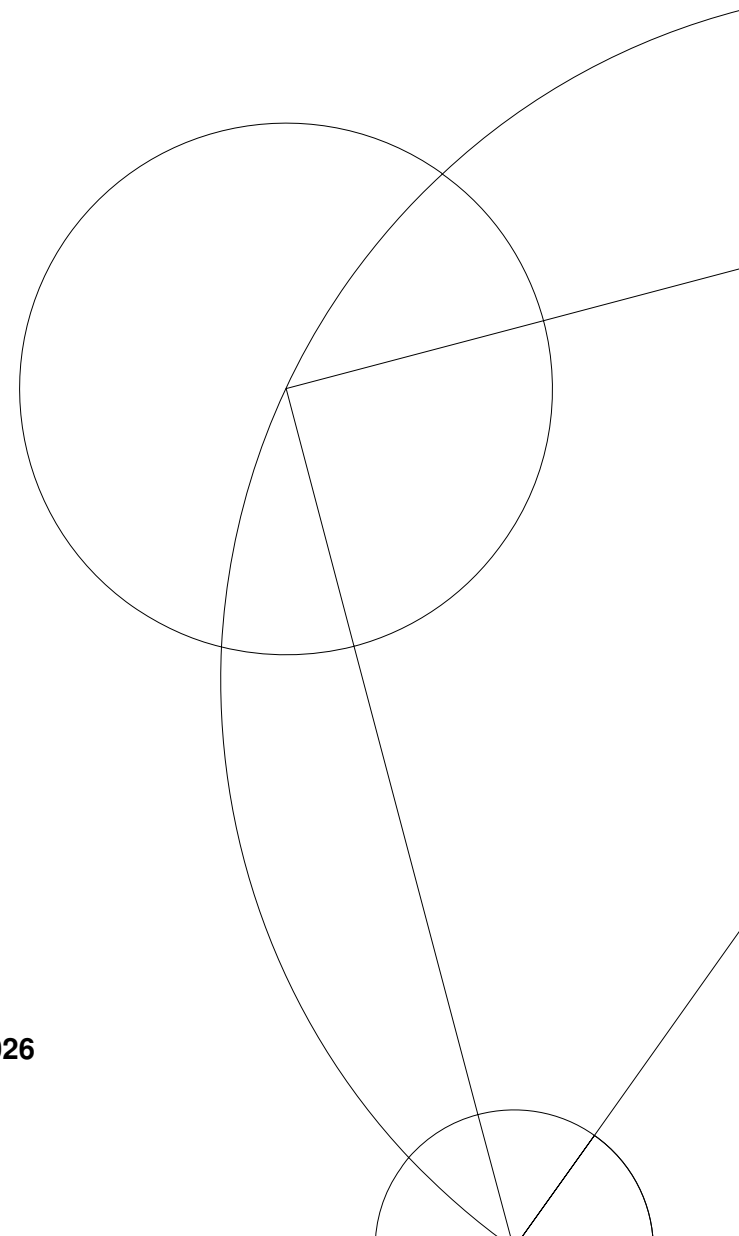
Project

Project Group 9

A hidden Markov model of visual attention

Exam project for the course Models for Complex Systems



Course Coordinator: Sebastian Weichwald

Department of Mathematical Sciences March 31, 2026

Group Members

Navn	KUid
Simon H. Hvidtfeldt	zgh166
Emil K. Vang	dkz250
Simon Henriksen	xwj436
William S. Saugmann	sgz655

Contents

Part I: A generative model

In this section, we establish the generative model as a dynamic Bayesian network, in particular, as a Hidden Markov Model (HMM). The aim is to describe and simulate both the hidden states and the observed neural spike counts in a probabilistic framework.
The model assumes time homogeneity, meaning that the transition probabilities between hidden states remain constant over time. As a consequence, the temporal variables can be fully described by a two-time-slice Bayesian network (2-TBN), where dependencies are specified only between time steps and repeated across the entire sequence.

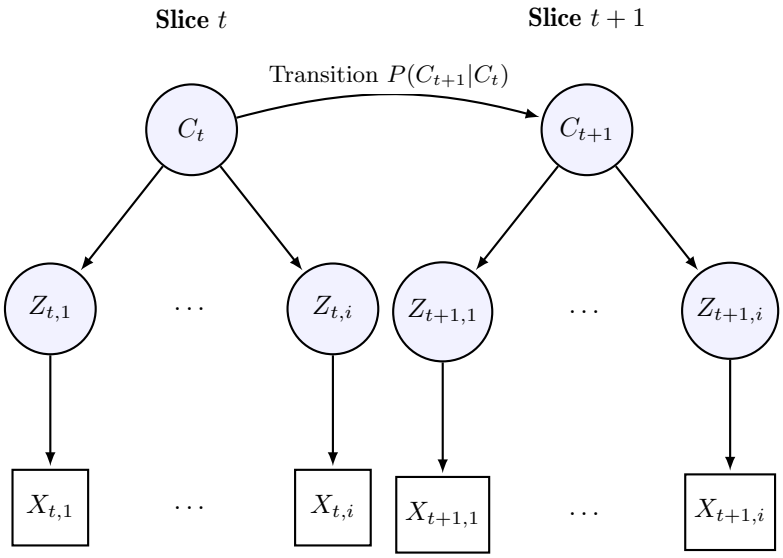


Figure 1: Graphical model representing two time slices (t and $t+1$). Hidden states are blue circles, and observed data are white squares.

The global hidden variables C_t follow a Markov assumption, implying that the future is conditionally independent of the past given the present state,

$$(C_{t+1} \perp C_{1:t-1} \mid C_t).$$

Thus, the process evolves sequentially through time, with each state depending only on its immediate predecessor.
Similarly, the local hidden variables $Z_{t,i}$ and the observed variables $X_{t,i}$ satisfy conditional independence properties derived from the Bayesian network via d-separation. In particular,

$$X_{t,i} \perp X_{1:t-1} \mid Z_{t,i},$$

and

$$Z_{t,i} \perp \{C_{1:t-1}, Z_{1:t-1}, X_{1:t-1}\} \mid C_t.$$

This reflects the hierarchical structure of the model, where C_t governs the distribution of $Z_{t,i}$, which in turn generates the observations $X_{t,i}$.
Taken together, these assumptions yield the following factorization of the joint distribution,

$$P(C_{1:T}, Z_{1:T,1:n}, X_{1:T,1:n}) = P(C_1) \prod_{t=1}^{T-1} P(C_{t+1} \mid C_t) \prod_{t=1}^{T} \prod_{i=1}^{n} P(Z_{t,i} \mid C_t) P(X_{t,i} \mid Z_{t,i}). \tag{1}$$

Having established the joint probability and the conditional independence, we now turn to the simulation of the model.

Visualization

The transition matrix describing the transition matrix specifies the probabilities of moving between hidden states and governs the temporal dynamics of the model. Under the Markov assumption, the

state at time $t+1$ depends only on the current state C_t, and the process evolves sequentially according to these transition probabilities.

Rather than analysing the transition dynamics in isolation, we illustrate their effect through simulated data. The following visualizations show how the hidden states evolve and how these transitions influence the observed spike activity.

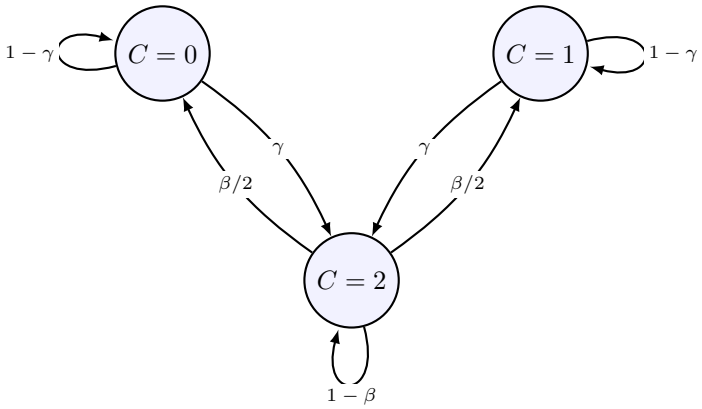


Figure 2: 3-state Markov chain for C_t with transition probabilities γ and β.

The system is initialized according to

$$P(C_1 = 2) = 1,$$

So the process always starts in the parallel processing state. From here, the model is simulated forward in time using the natural topological ordering

$$C_t \to Z_{t,i} \to X_{t,i}.$$

At each time step, the global state C_t is first sampled from the transition distribution, followed by the local attention variables $Z_{t,i}$, and finally the observed spike counts $X_{t,i}$.
There are different ways to define a topological order of execution. One way is to sample for each `t`, this approach is not unique. Properties of the HMM, namely the Markovian assumption, allow another order, isolating the C, Z, and X variables instead of each timestamp. This enables parallelism, which is beneficial from a computational standpoint.

We've chosen the more intuitive way, sampling with the following topological order,

$$C_t \to Z_{t,i} \to X_{t,i}$$

due to the scale of the assignment and simplicity.

Simulation

Algorithm 1 Simulate HMM Dataset

Require: T (time steps), n (neurons), $\alpha, \beta, \gamma, \lambda_0, \lambda_1$
Ensure: Sequences $C_{1:T}$, $Z_{1:T,1:n}$, $X_{1:T,1:n}$
Initialize random number generator
Compute transition matrix $\Gamma \leftarrow$ build_transition_matrix(β, γ)
Initialize arrays:
    $C \leftarrow$ vector of length T
    $Z \leftarrow$ matrix of size $T \times n$
    $X \leftarrow$ matrix of size $T \times n$
Set initial hidden state:
    $C_1 \leftarrow 2$
Sample initial latent and observed variables:
    $Z_1 \leftarrow$ sample_z_row(C_1, n, α)
    $X_1 \leftarrow$ sample_x_row$(Z_1, \lambda_0, \lambda_1)$
for $t = 2$ to T **do**
        $C_t \leftarrow$ sample_next_c(C_{t-1}, Γ)
        $Z_t \leftarrow$ sample_z_row(C_t, n, α)
        $X_t \leftarrow$ sample_x_row$(Z_t, \lambda_0, \lambda_1)$
end for
return C, Z, X

The function first initializes the random number generator and constructs the transition matrix. Three arrays are then created, C for the global hidden states, Z for the neuron attention assignments, X for the observed spike counts. The project specifies the initial condition

$$P(C_1 = 2) = 1$$

meaning that the system always starts in the parallel processing state. Therefore we set

$$C_1 = 2$$

and then sample the first row of Z and X. After this initialization, the simulation proceeds forward in time. For each time step: sample C_t from the Markov transition matrix, sample Z_t conditioned on C_t, sample X_t conditioned on Z_t. This process is repeated until all T time steps have been generated. For our initial test simulation, we use the parameter values suggested in the project description:

```
T = 100
n = 10
alpha = 0.9
beta = 0.2
gamma = 0.1
lambda0 = 1
lambda1 = 5
```

These values produce a simulation of 100 time steps with 10 neurons and allow us to generate example spike count data for visualization and later inference experiments.

Figure 3: Simulated spike counts for each neuron over time. Colours indicate the hidden state C_t, while markers show whether a neuron is attending ($Z_{t,i} = 1$) or not ($Z_{t,i} = 0$).

As seen by Figure 3, the spike sequence for individual neurons illustrates the hierarchical structure of the model. When a neuron is attending ($Z_{t,i} = 1$), higher spike counts are observed, whereas non-attending states ($Z_{t,i} = 0$) result in lower activity levels.

This demonstrates how the global hidden state C_t influences the observations indirectly through the local latent variables $Z_{t,i}$. The figure also highlights the variability across neurons, particularly in the parallel state, where multiple neurons can be active simultaneously.

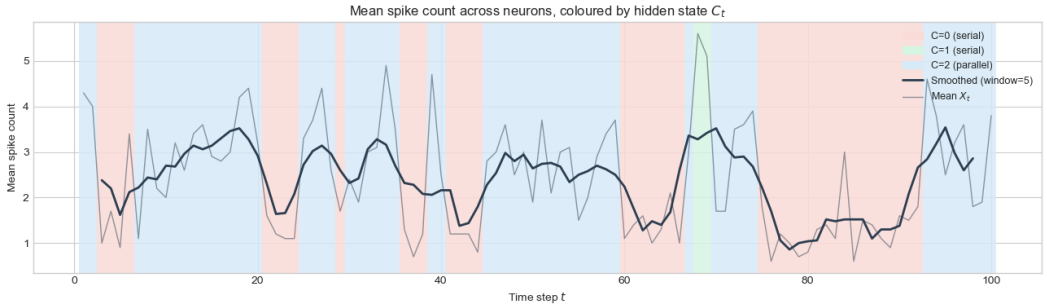


Figure 4: Mean spike count across neurons over time, coloured by the hidden state C_t. The black curve shows a smoothed version of the mean activity.

Figure 4 illustrates the mean spike count across neurons as a function of time, with colours indicating the underlying hidden state C_t. Clear differences between the states are observed. The serial states ($C_t = 0, 1$) exhibit more stable and distinct activity levels, whereas the parallel state ($C_t = 2$) produces greater variability in the observations.

The smoothed curve is computed using a moving average (window size 5) of the mean spike counts to reduce noise and emphasize temporal trends.

Multiclass logistic regression for decoding C_t

To complement the generative model, we fit a multiclass logistic regression model as a simple discriminative approximation of $P(C_t \mid X)$. The model is trained on simulated data, where each time point t is treated as a supervised example with label C_t. In the feature construction, the full observation sequence $X_{1:T}$ is used together with a normalized time index. This allows the classifier to exploit global patterns in the spike data when predicting the hidden state. The resulting model achieves an accuracy of approximately 0.48, which is above chance level for a three-class problem but still reflects substantial overlap between the classes. In particular, the parallel processing state $C_t = 2$ is more difficult to identify, as it produces more diverse observation patterns across neurons compared to the more structured serial states. It should be noted that using the full sequence $X_{1:T}$ introduces a deviation from the temporal structure of the HMM, as future observations are used when predicting C_t. Consequently, the logistic regression model should be interpreted as a purely discriminative approximation of the smoothing distribution $P(C_t \mid X_{1:T})$, rather than a model consistent with the generative assumptions. Overall, this experiment serves as a baseline, illustrating that the observations contain information about the hidden state, while also highlighting the limitations of a non-structured approach compared to the probabilistic inference methods developed in the following part.

Part II: Inference of hidden nodes

To compute the conditional probabilities of hidden variables, C_t, and the local hidden, $Z_{t,i}$, given the observed variables, $X_{1:T}$.

The previously defined joint distribution allows us to do effective message passing instead of creating large intermediate factors.

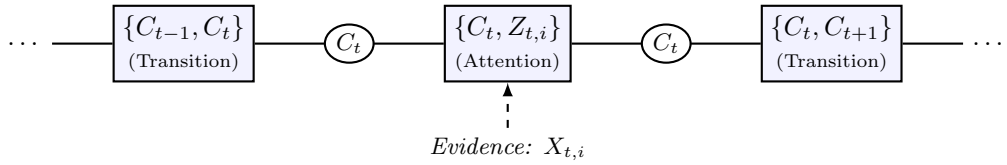
A method to compute exact inference we transform the HMM into a cluster tree and validate possible clique trees is the belief propagation algorithm.

For a cluster tree to be valid for belief propagation, it must obey the Family Preservation Property, ensuring that each factor in the joint distribution is contained within the scope of at least one cluster. In our model, this is naturally satisfied by the parent-child relations in the HMM.

From the cluster tree/graph, we define a clique tree which obeys the running intersection property, saying if X is contained in clusters, then every unique path from C_i to C_j has to contain X. This implies the separation sets, which are defined as the intersection between cluster C_i and C_j. This allows the intersection variables to be the message from one cluster to another.

With these things stated, we can dive into the algorithm specifics.

First, a root needs to be picked to initialize the upstream and downstream flows. We have chosen to visualize the model as a clique tree, as this structure directly supports the Running Intersection Property and forms the framework for our Belief Update implementation. We have designated C_T as the root node, which allows an intuitive division into a chronological forward-pass (evidence collection) and a backward-pass (smoothing), which ensures full calibration of the network.



Since the observed variables $X_{t,i}$ represent fixed evidence, they are not explicitly represented as nodes in the clique tree. Instead, the conditional probability distributions $P(X_{t,i}|Z_{t,i})$ are reduced to local factors and absorbed directly into the potentials of the attention cliques, $\psi_{\{C_t, Z_{t,i}\}}$, before the message-passing process. This reduction simplifies the graph structure and ensures that messages are only passed between the necessary latent variables.

The scheduling of our clique tree and the associated message-passing order is dictated by the principle of ready cliques. A clique is defined as being ready to send a message once it has received incoming messages from all of its neighbors except the recipient.

This systematic ordering ensures that each message is "complete" and contains the entire information from its branch of the network before it is normalized. By adhering to the ready protocol in

both the collect and distribute phases, we guarantee that the final clique beliefs are mathematically consistent and fully calibrated.

Implementation of Inference Algorithms

We only observe the spike counts $X_{t,i}$ from neurons, which act as our known evidence. We want to solve the inference problem by calculating the conditional probabilities of the hidden variables given the evidence

$$P(C_t|X) \quad \text{and} \quad P(Z_{t,i}|X)$$

Our model tracks many variables over $T = 100$ time steps. Trying to calculate these probabilities by marginalizing out all other variables would take too long. To solve this, we use discrete message passing with 2D and 3D arrays on a Clique Tree. Specifically, we use Algorithm 10.3 (Calibration using belief propagation in clique tree) from PGM.

To make the Python code run faster, we simplify the formal clique tree from the previous section. We do not create separate cliques for the attention variables in our code. Instead, we calculate their probabilities first and save them in a single evidence array (E).

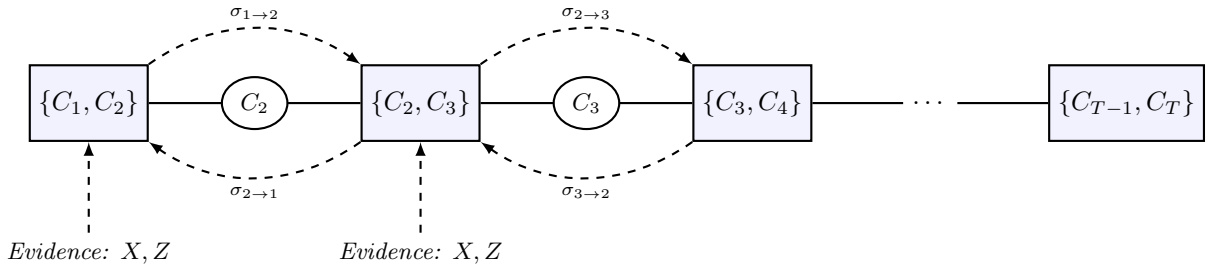


Figure 5: Simplified Clique Tree.

The algorithm processes this structure in four main steps:

1. **Initialization:** We start by breaking the time series model down into overlapping cliques for adjacent states $\{C_t, C_{t+1}\}$. We set up our initial potentials using 3×3 transition matrix Γ and the local observation evidence $P(X_t|C_t)$. When we condition on X_t we turn our observations into fixed numbers.

2. **Forward Pass:** We pass messages forward through the time steps from $t = 1$ to T. At each step, we sum out the past state C_{t-1}, normalize the message, and save it in an array.

3. **Backward Pass:** we pass messages backward from the end of the sequence. To avoid counting evidence twice, we divide the incoming backward message by the forward message saved during Forward Pass.

4. **Marginals:** Once the tree is fully calibrated, we get the exact clique beliefs by summing out the extra variables from our cliques. Finally, we push this probability down to the individual neurons to compute the attention probabilities $P(Z_{t,i}|X)$.

Test of Inference Model

When testing our inference model, we had two different approaches. A visual test and a mathematical test.

Visual

For the visual test, we generated a dataset where we already know the true hidden states. Then, we run our model using only the observed spike counts $\mathbf{X}$. We then plot the probabilities the model guesses for the parallel state $C_t = 2$ right next to the actual state to see if they match up.

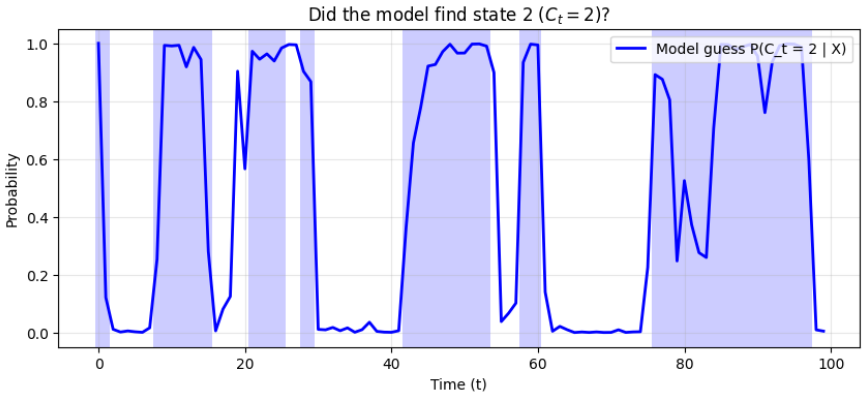


Figure 6: Visual test of Inference model.

As shown in Figure 6, we can see that the inference algorithm successfully tracks the hidden states with high confidence. This can be seen by the sharp transitions (the blue curve), which align with the true state (blue shaded region). However, around $t = 80$, the model's certainty drops. While the visual test gives us a good intuition, we can also test the inference model by running many simulations and verifying the empirical averages.

Mathematical

We can observe that the difference between the true state ($C_t = c$), and our predicted probability $P(C_t = c|X)$ should have a mean of zero.

$$\frac{1}{N}\sum_{k=1}^{N}\left(\mathbf{1}(C_t^{(k)} = c) - P(C_t = c|X^{(k)})\right) \approx 0$$

Where $C_t^{(k)}$ is the actual true state at time t during simulation number k, and $X^{(k)}$ is the dataset we generated during simulation number k. By running many simulations, we can compute these probabilities with our inference algorithm and check if the average difference is zero, or close to.

We simulated 100 independent experiments and computed the probabilities with our implemented inference algorithm. The test confirms what the visual test seems to indicate.

Variable	State (c or z)	Average Error
C_t	0	-0.00071
C_t	1	-0.00364
C_t	2	0.00435
$Z_{t,i}$	0	0.00132
$Z_{t,i}$	1	-0.00132

Table 1: Results of the zero-mean simulation check across 100 runs.

As can be seen by Table 1 for the main state C_t, the average differences were very close to zero (State 0: -0.00071, State 1: -0.00364, and State 2: 0.00435). The differences for the individual neuron attention variables $Z_{t,i}$ were also practically zero (0.00132 and -0.00132).

Applying Inference Algorithm to Real Data

After validating our algorithm, we applied it to the 10 real datasets provided in the data folder `proj_HMM`. For each experiment, we computed the probability of the brain being in a parallel processing state ($P(C_t = 2)$) or in a serial processing state ($P(C_t \in \{0, 1\})$).

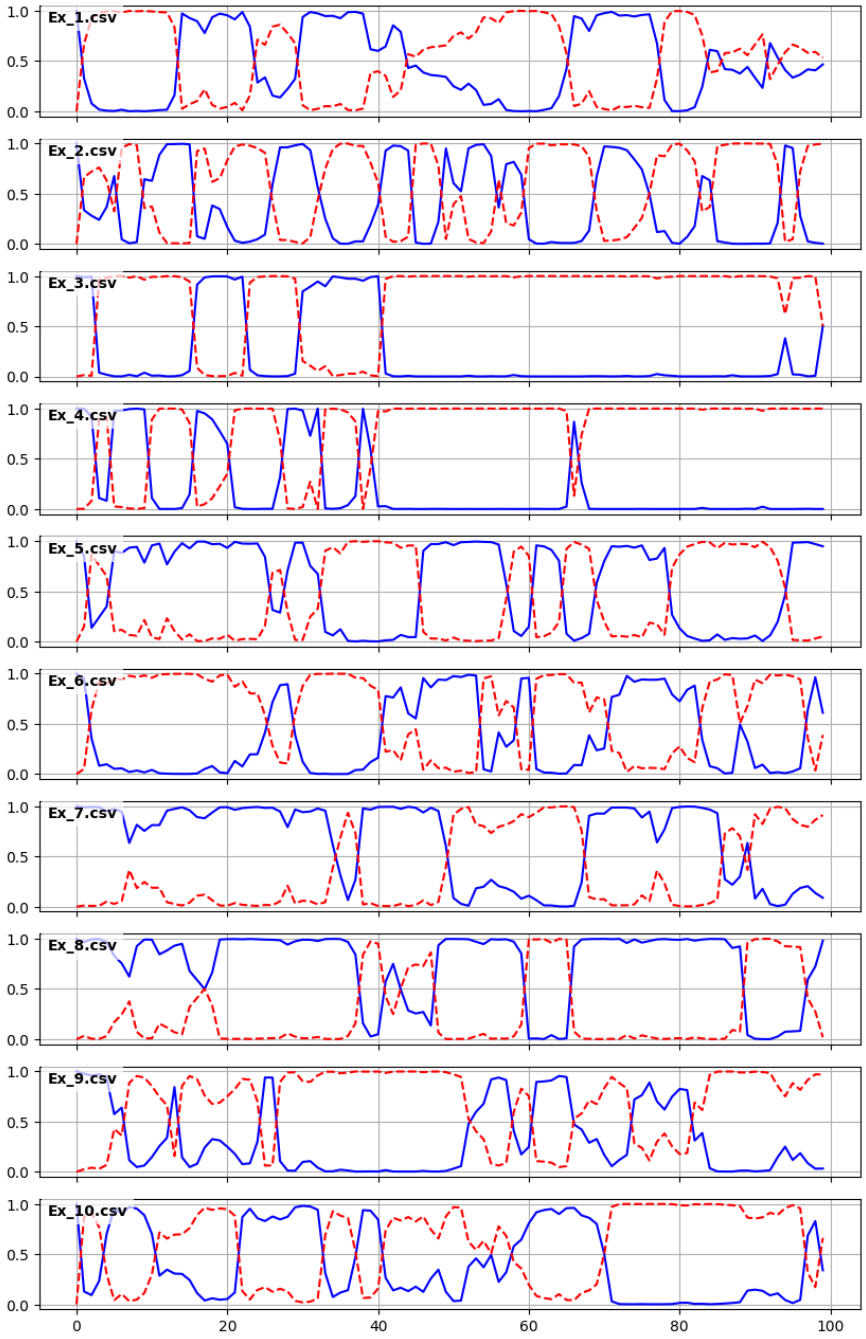


Figure 7: Inferred Probabilities for Parallel vs. Serial Processing

Figure 7 shows the inferred probabilities of the network being in a parallel state (the solid blue line, $C_t = 2$) compared to a serial state (the dashed red line, $C_t \in \{0, 1\}$) across 10 real datasets.

As seen in the plots, every trial starts in the parallel state. At exactly $t = 1$, the probability (the blue line) is exactly 1.0. This matches our setup, where we forced the model to start in state 2 ($P(C_1 = 2) = 1$). Throughout the trial, the model is very sure of its predictions. The probabilities mostly stay close to 1.0 or 0 instead of staying around 0.5. This tells us the neural spike data is clear enough for the algorithm to find the correct network state. Also, the network constantly switches between parallel and serial processing, and the time spent in each state is very different across the ten datasets. Finally, the probabilities of the parallel and serial states perfectly mirror each other.

Because states 0 and 1 are combined to represent the serial state, their total probabilities must always sum to 1, together with state 2. Seeing this exact mirror effect confirms that our message-passing algorithm normalizes the probabilities correctly.

Part III: Learning of the parameters

In the final part of our project, we want to learn the biological parameters $\alpha, \beta, \gamma, \lambda_0$, and λ_1 directly from the data.

Parameter estimation with fully observed observations

First, we need to find out how to learn the parameters if we pretend we have complete data. This means we assume we can observe all the variables i.e. the overall state, C_t, the attention variables $Z_{t,i}$, and the spike counts $X_{t,i}$.

Because the data is fully observed, the Bayesian network decomposes completely into independent local likelihoods for each Conditional Probability Distribution (CPD). If this wasn't the case, then it would be much more complex to optimize as variables would be conditionally dependent and affect each other. This implies the global likelihood,

$$P(\mathbf{C}, \mathbf{Z}, \mathbf{X} \mid \theta) = P(C_1) \prod_{t=2}^{T} P(C_t \mid C_{t-1}, \beta, \gamma) \prod_{t=1}^{T} \prod_{i=1}^{N} P(Z_{t,i} \mid C_t, \alpha) \prod_{t=1}^{T} \prod_{i=1}^{N} P(X_{t,i} \mid Z_{t,i}, \lambda_0, \lambda_1) \quad (2)$$

- The transition parameters (β, γ) only appear in the first sum.
- The coupling parameter α only appears in the second sum.
- The emission parameters (λ_0, λ_1) only appear in the third sum.

Because these sets of parameters are independent, we can maximize the global likelihood by maximizing each sum independently. This allows us to estimate the parameters for the transition model and the observation model using *sufficient statistics*, which are the empirical calculations of averages and relative frequencies:

λ_0 and λ_1: We calculate the empirical mean of the spike counts $\mathbf{X}$ for the times when $Z = 0$ and $Z = 1$ respectively.

α: We calculate the relative frequency of the neuron's state matching the monkeys overall state. We count how often $Z_{t,i} = C_t$ during the times the monkey is in a serial state i.e. $C_t \in \{0, 1\}$.

β and γ: We calculate the relative frequency of the monkey switching between the parallel state, $C_t = 2$, and the serial states, $C_t \in \{0, 1\}$.

Parameter Estimation with Partial Observations

In the partially observed scenario, the attention and state variables ($Z_{t,i}$ and C_t) are hidden, and $X_{t,i}$ are observed. The introduction of hidden variables breaks the global likelihood decomposition, unimodality, and closed-form representation, which results in a highly complex, multimodal likelihood function where analytical MLE is no longer an option.

The log-likelihood takes the form:

$$\ell(\theta : \mathcal{D}) = \sum_{t,i} \log \sum_{c,z} P(x_{t,i}, z_{t,i}, c_t \mid \theta) \quad (3)$$

Because the summation over hidden states occurs *inside* the logarithm, the likelihood no longer decomposes into independent local terms for each parameter. This coupling of parameters means that the likelihood is *multimodal*

To formally treat the missing data in our HMM, we adopt the *observability model*. We define a set of binary observability indicators $\mathbf{O}$, where $O_X = 1$ denotes that the spike counts are always observed, and $O_C = 0, O_Z = 0$ denotes that the state and attention variables are latent.

In our case, the observability pattern is constant across all trials. We assume that the data is *Missing at Random* (MAR), meaning:

$$P(\mathbf{O} \mid \mathbf{X}_{obs}, \mathbf{X}_{miss}) = P(\mathbf{O} \mid \mathbf{X}_{obs}) \tag{4}$$

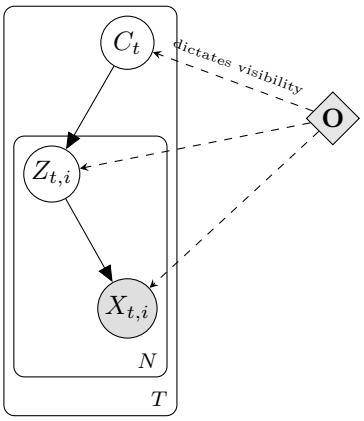


Figure 8: Visualization of Missing at Random (MAR) in the HMM model. The white nodes $(C_t, Z_{t,i})$ are latent, while the gray node $(X_{t,i})$ is observed. The observation mechanism $\mathbf{O}$ is independent of the latent values.

also represented as:

$$P(\mathbf{O} \mid \mathbf{X_{ti}}_{obs}, \mathbf{C_t}_{miss}, \mathbf{Z_{ti}}_{miss}) = P(\mathbf{O} \mid \mathbf{X_{ti}}_{obs}) \tag{5}$$

Since the mechanism $\mathbf{O}$ is independent of the latent values $\mathbf{X}_{miss} = \{C, Z\}$ and our parameters θ, the observability model is considered ignorable. This theoretical justification allows us to focus solely on maximizing the incomplete data likelihood:

$$L(\theta : \mathbf{X}_{obs}) = \sum_{\mathbf{X}_{miss}} P(\mathbf{X}_{obs}, \mathbf{X}_{miss} \mid \theta) \tag{6}$$

This simplification is what enables the use of the Expectation-Maximization algorithm for parameter estimation.

One other key concept within partially observed scenarios is identifiability. Imagine that your EM algorithm finds that state 0 has $\lambda_0 = 10$ and state 1 has $\lambda_1 = 2$. If you swap all labels (calling 0 for 1 and vice versa) and swap the rates, your likelihood will be the same. This shows that the HMM is theoretically non-identifiable. This should be addressed in practice through informed initialization and parameter constraints in the EM algorithm. This means that we choose one of the many possible (but mathematically equivalent) solutions so we can interpret the parameters and rely on these probabilities.

Test of Implementation on Simulated Data

To check if our code for learning parameters works, we tested it on some simulated data. First, we generated a large dataset with $T = 10000$ time steps using our `simulate_hmm` function. We used the true parameters given in the project description: $\alpha = 0.9$, $\beta = 0.2$, $\gamma = 0.1$, $\lambda_0 = 1$, and $\lambda_1 = 5$.

We then gave this complete simulated data to our `learn_parameters_complete` function to see if it could find the true parameters again.

The function worked well and gave us these estimated values: $\hat{\alpha} = 0.898$, $\hat{\beta} = 0.195$, $\hat{\gamma} = 0.098$, $\hat{\lambda}_0 = 1.000$, and $\hat{\lambda}_1 = 4.995$. Because these numbers are very close to the true parameters, it shows that our counting method works correctly.

Parameter	True Value	Learned Value
α	0.9	0.898
β	0.2	0.195
γ	0.1	0.098
λ_0	1	1.000
λ_1	5	4.995

Table 2: Parameter Recovery Test Results

Learning with only X observed

In real experiments, we cannot actually see the monkey's cognitive state (C_t) or the attention states of the individual neurons ($Z_{t,i}$). We only get to see the spike counts, $\mathbf{X}$. Since these important variables are hidden from us, this becomes a learning problem with partial observations.

When we have complete data, the math is easy because we can just count the occurrences to find the best parameters. But when variables are hidden, we cannot just count anymore because everything gets mixed. There is no simple formula to find the best parameters directly.

As previously discussed, hidden variables also introduce the theoretical problem of "identifiability". To work around this in practice, we have to give our algorithm a good starting guess so it chooses the one mathematically equivalent solution that actually makes biological sense.

Because we only observe $\mathbf{X}$, we will be using the hard-assignment Expectation-Maximization (EM) algorithm which operates in a discrete space compared the to soft EM (continues). This is a loop that repeats two steps over and over until the parameters stop changing:

1. **The Expectation-step (E-step):** We use our current guess for the parameters $(\alpha, \beta, \gamma, \lambda_0, \lambda_1)$ and the inference code we wrote in Part II to guess the hidden states. Instead of working with probabilities, this "hard" version of EM forces the model to pick the single most likely state for each hidden variable. We calculate this as:

$$\hat{C}_t = \arg\max_c P(C_t = c \mid \mathbf{X} = x)$$

$$\hat{Z}_{t,i} = \arg\max_z P(Z_{t,i} = z \mid \mathbf{X} = x)$$

2. **The Maximization-step (M-step):** Now, we just pretend that our hard guesses for $\hat{C}_1, \ldots, \hat{C}_T$ and $\hat{Z}$ are true observations. This gives us a "complete" dataset, so we can update the parameters using the exact same simple counting method we used for complete data earlier.

By repeating these two steps, the algorithm improves the parameters. When the numbers finally stop changing, it means the algorithm has converged, and we have successfully learned the biological parameters using only the raw spike counts.

Test on Simulated Data

We tested our EM algorithm on simulated data ($T = 5000$) to see how bad our starting guess could be before the algorithm failed.

First, we tried a very bad guess: $\alpha = 0.5, \beta = 0.5, \gamma = 0.5, \lambda_0 = 2.0$, and $\lambda_1 = 4.0$. We only made sure that λ_0 was smaller than λ_1 to stop the algorithm from swapping the hidden state labels. Table 3 and Figure 9 show how the parameters broke down during this test.

Parameter	True Value	Initial Guess	Learned Value
α	0.9	0.5	1.000
β	0.2	0.5	0.000
γ	0.1	0.5	0.000
λ_0	1.0	2.0	3.089
λ_1	5.0	4.0	`NaN`

Table 3: Parameter values showing the failure of the EM algorithm to converge.

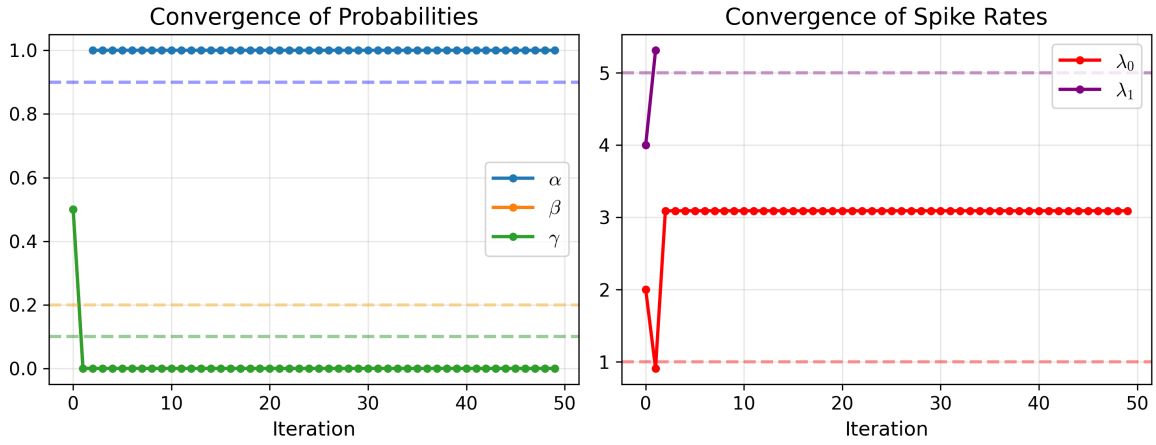


Figure 9: Plot showing the EM algorithm failing to converge within 50 iterations with a completely unstructured guess.

As seen in the results, the algorithm failed to converge. Instead of finding the true parameters, it ran for the maximum limit of 50 iterations without settling on meaningful values. This happened because setting $\alpha = 0.5$ tells the model that the neurons' attention is just a random 50/50 coin flip. This completely breaks the link between the monkey's true brain state and the neuron spikes. Without this link, the algorithm cannot learn anything, and the model fails to optimize the parameters.
To fix this, we used a guess that was still bad, but had a little more structure: $\alpha = 0.6, \beta = 0.4, \gamma = 0.4, \lambda_0 = 2.0$, and $\lambda_1 = 4.0$. This guess wrongly assumes the monkey switches states very often, and it puts the two firing rates too close together. Table 4 compares our learned values to the true values, and Figure 10 shows how the parameters converged.

Parameter	True Value	Initial Guess	Learned Value
α	0.9	0.6	0.904
β	0.2	0.4	0.175
γ	0.1	0.4	0.100
λ_0	1.0	2.0	0.923
λ_1	5.0	4.0	5.205

Table 4: Comparison of true parameters, initial guesses, and learned parameters after 7 iterations.

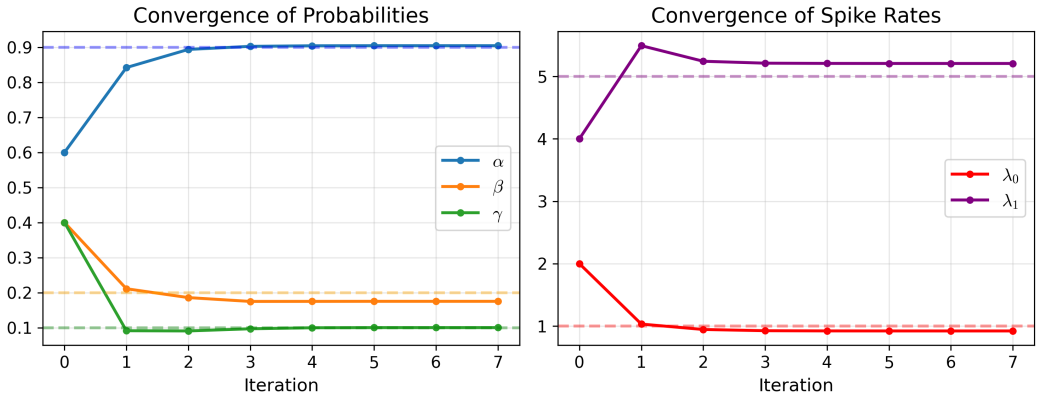


Figure 10: Plot showing the successful convergence of the parameters over 7 iterations.

Because α is greater than 0.5, the algorithm worked. In just 7 iterations, it pushed the bad guesses close to the true values. The final learned values were $\hat{\alpha} = 0.904$, $\hat{\beta} = 0.175$, $\hat{\gamma} = 0.1$, $\hat{\lambda}_0 = 0.923$, and $\hat{\lambda}_1 = 5.205$. This proves our code is robust and can successfully recover the parameters even with a poor starting point, provided the initial guess retains some structural validity.

Test on Real Data

In the simulated data, we knew the true hidden states of the monkey. But in the real data, we only have the raw spike counts $\mathbf{X}$. The goal here is to use the hard-assignment EM algorithm on this real data to find the transition probabilities, which are α, β, γ, and the firing rates λ_0 and λ_1.
To get a clear overview of the dataset, Figure 11 shows the mean spike count across all neurons of each of the 10 experiments.

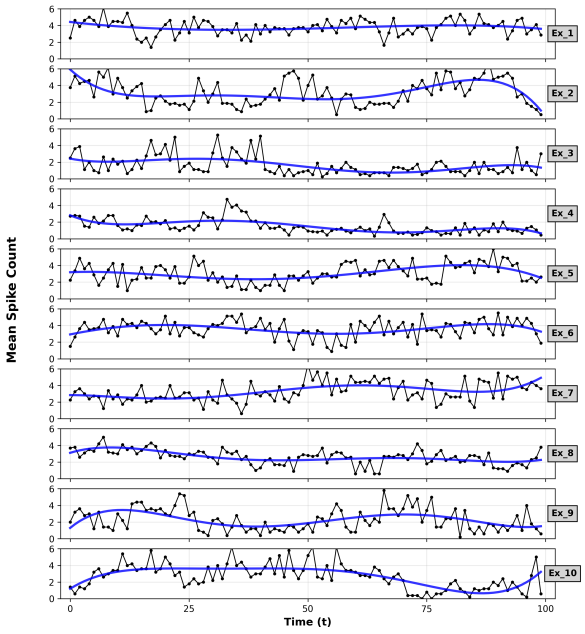


Figure 11: Mean spike count for all 10 real experiments over 100 time intervals.

As Figure 11 shows, the real neural data is noisy. However, we can already see clear waves of high and low activity. This is the noise our EM algorithm needs to clean up to figure out when the monkey is actually paying attention.

We ran the hard-assignment EM algorithm on all the real data files. We started each run with the same guess i.e. $\alpha = 0.6, \beta = 0.4, \gamma = 0.4, \lambda_0 = 2.0, \lambda_1 = 4.0$.

Experiment	**Iterations**	α	β	γ	λ_0	λ_1
Ex_1.csv	Failed	1.000	0.000	0.000	3.780	NaN
Ex_2.csv	14	0.951	0.174	0.132	1.489	6.096
Ex_3.csv	4	0.954	0.143	0.026	0.911	6.061
Ex_4.csv	12	0.891	0.200	0.072	0.503	4.136
Ex_5.csv	7	0.862	0.082	0.132	1.051	5.032
Ex_6.csv	8	0.820	0.200	0.036	0.728	4.552
Ex_7.csv	8	0.716	0.500	0.033	1.314	5.819
Ex_8.csv	10	0.766	0.091	0.114	0.656	4.753
Ex_9.csv	9	0.950	0.281	0.119	1.028	4.525
Ex_10.csv	10	0.815	0.238	0.051	0.521	4.644

Table 5: Estimated parameters and convergence time for the 10 real-data experiments.

Looking at table 5, we can see a pattern in most of the files. The algorithm usually finds a difference between the background noise state λ_0 and the attention state λ_1. Across almost all successful runs, λ_0 is low, typically around 0.5 to 1.5 spikes. In contrast, λ_1 is higher, ranging from 4 to 6 spikes. We can also see that the transition probability α is generally high. Mostly above 0.8. This tells us that the monkey's cognitive states are stable. When the monkey pays attention, it keeps paying attention for a while instead of switching every millisecond.

An example of this working is `Ex_2.csv`, as visualized in Figure 12.

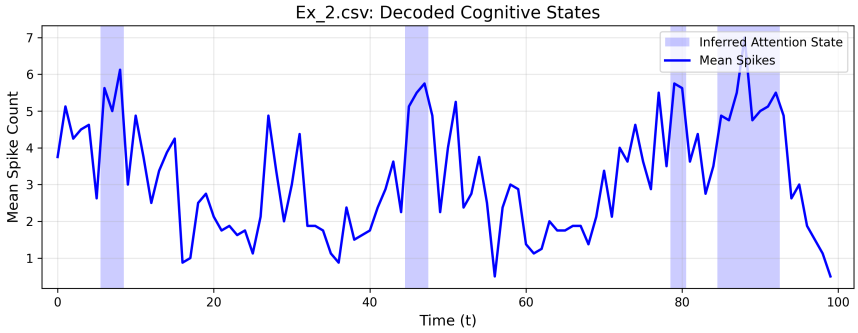


Figure 12: Successful decoding of `Ex_2.csv`.

The algorithm took 14 iterations to settle on the final numbers, finding a clear difference between the background state, $\lambda_0 = 1.489$, and the active attention state, $\lambda_1 = 6.096$. In the plot, the blue line is the mean spike count, and the light blue shaded areas show when the algorithm estimates that the monkey is paying attention. We can see that these shaded areas match the high peaks in the data. Also, the high α value of 0.951 means that the attention state is stable. This can also be seen by the figure. The light blue areas form blocks, instead of jumping back and forth between the two states.

On the other hand, `Ex_1.csv` shows what happens when the model fails. If we look at the raw data from Figure 11 for this experiment, there are no clear waves of high and low activity.

Our hard-assigmnent EM algorithm always looks for two different states, but because the data in `Ex_1.csv` isn't clear, the model struggles and ends up putting all the time steps into one single state. This leaves the second state empty. When we try to calculate the average spike count for an empty state, it divides by zero. This is the reason we get `NaN` for λ_1 in Table 5, and the reason we can't display any inferred attention states as we did for `Ex_2.csv`. See Figure 13.

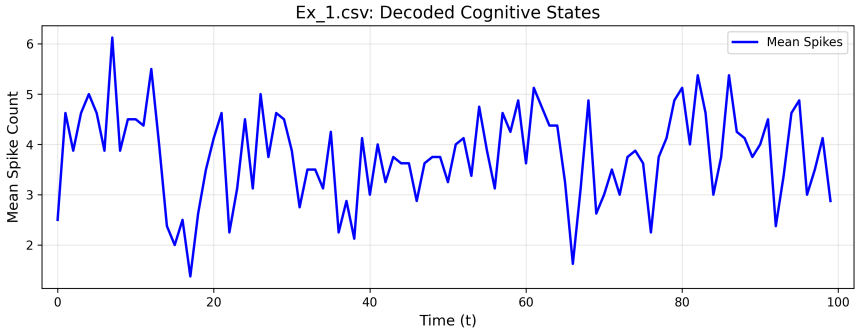


Figure 13: Failed decoding of `Ex_1.csv`.

Appendix

Python code related to part I:

Parameter setup

```
T = 100
n = 10
alpha = 0.9
beta = 0.2
gamma = 0.1
lambda0 = 1
lambda1 = 5
```

Simulation function

```
def build_transition_matrix(beta, gamma):

    Gamma = np.array([
        [1 - gamma, 0,         gamma],
        [0,         1 - gamma, gamma],
        [beta / 2,  beta / 2,  1 - beta]
    ])
    return Gamma


def sample_next_c(prev_c, Gamma, rng):

    return rng.choice([0, 1, 2], p=Gamma[prev_c])


def sample_z_row(c_t, n, alpha, rng):

    if c_t == 0:
        p = 1 - alpha
    elif c_t == 1:
        p = alpha
    else:  # c_t == 2
        p = 0.5

    return rng.binomial(1, p=p, size=n)


def sample_x_row(z_row, lambda0, lambda1, rng):
    rates = np.where(z_row == 0, lambda0, lambda1)
    return rng.poisson(lam=rates)


def simulate_hmm(T, n, alpha, beta, gamma, lambda0, lambda1, seed=None):

    rng = np.random.default_rng(seed)
    Gamma = build_transition_matrix(beta, gamma)

    C = np.zeros(T, dtype=int)
    Z = np.zeros((T, n), dtype=int)
    X = np.zeros((T, n), dtype=int)

    # Initial state: project specifies P(C1 = 2) = 1
    C[0] = 2

    # Sample Z and X for t = 0
    Z[0] = sample_z_row(C[0], n, alpha, rng)
    X[0] = sample_x_row(Z[0], lambda0, lambda1, rng)

    # Sample the remaining time points
    for t in range(1, T):
        C[t] = sample_next_c(C[t - 1], Gamma, rng)
        Z[t] = sample_z_row(C[t], n, alpha, rng)
        X[t] = sample_x_row(Z[t], lambda0, lambda1, rng)

    return C, Z, X
```

Logistic Regression

```
def generate_logistic_data_allX(
    num_simulations, T, n, alpha, beta, gamma, lambda0, lambda1, seed=None
    ):

    rng = np.random.default_rng(seed)

    X_data = []
    y_data = []

    for sim in range(num_simulations):
        sim_seed = rng.integers(0, 1_000_000_000)

        C, Z, X = simulate_hmm(
            T=T,
            n=n,
            alpha=alpha,
            beta=beta,
            gamma=gamma,
            lambda0=lambda0,
            lambda1=lambda1,
            seed=int(sim_seed)
        )

        # Flatten the full observation matrix X_{1:T}
        # X has shape (T, n), so X_flat has shape (T*n,)
        X_flat = X.reshape(-1)

        # Create one supervised example per time point t
        for t in range(T):
            t_feature = np.array([t / (T - 1)]) if T > 1 else np.array([0.0])
            features = np.concatenate([X_flat, t_feature])

            X_data.append(features)
            y_data.append(C[t])

    X_data = np.array(X_data)
    y_data = np.array(y_data)

    return X_data, y_data

    # Generate training data using ALL X
X_data, y_data = generate_logistic_data_allX(
    num_simulations=2000,
    T=T,
    n=n,
    alpha=alpha,
    beta=beta,
    gamma=gamma,
    lambda0=lambda0,
    lambda1=lambda1,
    seed=42
)


# Train/test split
X_train, X_test, y_train, y_test = train_test_split(
    X_data,
    y_data,
    test_size=0.2,
    random_state=42,
    stratify=y_data
)


# Logistic regression with scaling
model = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        solver="lbfgs",
        max_iter=2000, C = 10, random_state=42
    )
```

```
)


# Train model
model.fit(X_train, y_train)


# Predictions
y_pred = model.predict(X_test)
```

Python code related to part II:

Belief update method

```
def belief_update(X, alpha, beta_param, gamma_p, lambda_0, lambda_1):
    T, n = X.shape

    # Set up the probability tables (CPTs)
    Gamma = np.array([
        [1 - gamma_p, 0, gamma_p],
        [0, 1 - gamma_p, gamma_p],
        [beta_param / 2, beta_param / 2, 1 - beta_param]
    ])

    P_Z_given_C = np.array([
        [alpha, 1 - alpha],
        [1 - alpha, alpha],
        [0.5, 0.5]
    ])

    # Calculate the local evidence E[t, c] = P(X_t | C_t = c)
    E = np.zeros((T, 3))
    for t in range(T):
        for c in range(3):
            p_x_z0 = poisson.pmf(X[t], lambda_0)
            p_x_z1 = poisson.pmf(X[t], lambda_1)
            p_xi = p_x_z0 * P_Z_given_C[c, 0] + p_x_z1 * P_Z_given_C[c, 1]
            E[t, c] = np.prod(p_xi)

    # Set up the initial cliques (boxes for the variables)
    beta = np.zeros((T - 1, 3, 3))
    for t in range(T - 1):
        beta[t] = Gamma * E[t+1]

    prior = np.array([0.0, 0.0, 1.0])
    beta[0] = (beta[0].T * (prior * E[0])).T

    # UPWARD CYCLE (Forward pass)
    mu = np.ones((T - 1, 3)) # We need this to save the forward messages
    for t in range(T - 2):
        sigma = np.sum(beta[t], axis=0)
        sigma /= np.sum(sigma)

        beta[t+1] = (beta[t+1].T * sigma).T
        mu[t] = sigma # Save the message for later

    # Normalize at the very end of the chain
    sigma = np.sum(beta[-1], axis=0)
    sigma /= np.sum(sigma)
    mu[-1] = sigma

    # DOWNWARD CYCLE (Backward pass)
    sigma = np.sum(beta[-1], axis=0)
    sigma /= np.sum(sigma)

    for t in range(T - 2, -1, -1):
        # The magic division step to avoid double-counting the evidence:
        beta[t] *= (sigma / mu[t])

        sigma = np.sum(beta[t], axis=1)
        sigma /= np.sum(sigma)
```

```
    # Get the final probabilities (beliefs) for state C
    gamma_C = np.zeros((T, 3))
    for t in range(T - 1):
        gamma_C[t] = np.sum(beta[t], axis=1)
        gamma_C[t] /= np.sum(gamma_C[t])

    gamma_C[-1] = np.sum(beta[-1], axis=0)
    gamma_C[-1] /= np.sum(gamma_C[-1])

    # Push the probabilities down to get the beliefs for Z
    gamma_Z = np.zeros((T, n, 2))
    for t in range(T):
        for i in range(n):
            p_x_z0 = poisson.pmf(X[t, i], lambda_0)
            p_x_z1 = poisson.pmf(X[t, i], lambda_1)

            for c in range(3):
                unc_z0 = P_Z_given_C[c, 0] * p_x_z0
                unc_z1 = P_Z_given_C[c, 1] * p_x_z1
                norm_z = unc_z0 + unc_z1

                if norm_z > 0:
                    gamma_Z[t, i, 0] += (unc_z0 / norm_z) * gamma_C[t, c]
                    gamma_Z[t, i, 1] += (unc_z1 / norm_z) * gamma_C[t, c]

    return gamma_C, gamma_Z
```

Zero mean simulation test

```
N_replications = 100

# We will keep track of the total differences (or errors) here
diff_C = np.zeros(3)
diff_Z = np.zeros(2)

for _ in range(N_replications):
    # New test data
    C_sim, Z_sim, X_sim = simulate_hmm(T, n)

    # Let our model calculate the probabilities using only the spike counts (X)
    gamma_C, gamma_Z = belief_update(
        X_sim,
        alpha=0.9,
        beta_param=0.2,
        gamma_p=0.1,
        lambda_0=1,
        lambda_1=5
    )

    # Check how far off our model's guesses are for the main state C
    for c in range(3):
        # Make an array of 1s (if it's the real state) and 0s (if it's not)
        indicator_C = (C_sim == c).astype(float)
        # Find the average difference for this whole run and add it to our total
        diff_C[c] += np.mean(indicator_C - gamma_C[:, c])

    # Do the exact same check for the individual neurons (Z)
    for z in range(2):
        indicator_Z = (Z_sim == z).astype(float)
        # Find the average difference across all time steps and all 10 neurons
        diff_Z[z] += np.mean(indicator_Z - gamma_Z[:, :, z])

# Finally, divide by 100 to get the overall average difference across all runs
diff_C /= N_replications
diff_Z /= N_replications
```

Applying Inference to Real data

```
data_folder = 'proj_HMM'
```

```
# get all the .csv files in that folder
file_names = [f for f in os.listdir(data_folder) if f.endswith('.csv')]

# Sort them by the numbers in their names
# (otherwise file '10.csv' showed up right after 'ex_1.csv')
file_names.sort(key=lambda f: int(''.join(filter(str.isdigit, f))))

# Set up a figure with 10 stacked plots that share the same x-axis timeline
fig, axes = plt.subplots(10, 1, figsize=(10, 16), sharex=True, sharey=True)

# Go through each file and plot the results
for i, file_name in enumerate(file_names):
    file_path = os.path.join(data_folder, file_name)

    # Load the dataset
    df = pd.read_csv(file_path, sep=None, engine='python')

    # Clean up the data: drop the time column 't' if it's there,
    # so our X matrix only contains the spike counts for the 10 neurons
    if 't' in df.columns:
        X_real = df.drop(columns=['t']).values
    else:
        X_real = df.values

    T_real = X_real.shape[0]

    # Guess the hidden states using the real spike counts
    gamma_C_real, _ = belief_update(
        X=X_real,
        alpha=0.9,
        beta_param=0.2,
        gamma_p=0.1,
        lambda_0=1,
        lambda_1=5
    )

    # Plot the probabilities for Parallel (state 2) vs Serial (states 0 and 1
        combined)
    axes[i].plot(np.arange(T_real), gamma_C_real[:, 2], label='P(C_t=2) [Parallel]'
        , color='blue', lw=1.5)
    axes[i].plot(np.arange(T_real), gamma_C_real[:, 0] + gamma_C_real[:, 1], label=
        'P(C_t in {0,1}) [Serial]', color='red', linestyle='--', lw=1.5)

    # Put the filename directly inside the plot corner
    axes[i].text(0.01, 0.85, file_name, transform=axes[i].transAxes, fontsize=10,
        fontweight='bold', bbox=dict(facecolor='white', alpha=0.7, edgecolor='none'
        ))
    axes[i].grid(True)
```

Python code related to part III:

Mazimum Likelihood Estimation

```
def learn_parameters_complete(C, Z, X):
    # Find the lambdas (the average spike rates)
    lambda_0_hat = np.mean(X[Z == 0])
    lambda_1_hat = np.mean(X[Z == 1])

    # Find alpha (how often the neuron matches the monkey's state)
    # First, we only care about the times the monkey is actually in a serial state
        (0 or 1)
    serial_mask = (C == 0) | (C == 1)
    C_serial = C[serial_mask]
    Z_serial = Z[serial_mask]

    # Check how often Z is exactly the same as C, and take the average
    matches = (Z_serial == C_serial[:, np.newaxis])
    alpha_hat = np.mean(matches)

    # Find out beta and gamma by counting the state changes
    transitions_from_2 = 0
    transitions_2_to_serial = 0
```

```
    transitions_from_serial = 0
    transitions_serial_to_2 = 0

    for t in range(len(C) - 1):
        if C[t] == 2:
            transitions_from_2 += 1
            # Did it switch to serial?
            if C[t+1] in [0, 1]:
                transitions_2_to_serial += 1
        elif C[t] in [0, 1]:
            transitions_from_serial += 1
            # Did it switch to parallel?
            if C[t+1] == 2:
                transitions_serial_to_2 += 1

    # Turn our counts into actual probabilities (percentages)
    beta_hat = transitions_2_to_serial / transitions_from_2 if transitions_from_2 >
         0 else 0.0
    gamma_hat = transitions_serial_to_2 / transitions_from_serial if
        transitions_from_serial > 0 else 0.0

    return alpha_hat, beta_hat, gamma_hat, lambda_0_hat, lambda_1_hat
```

EM Algorithm

```
def hard_em(X, init_params, max_iters=50, tol=1e-4):
    # Unpack the rough guesses we are starting with
    alpha, beta, gamma_p, lambda_0, lambda_1 = init_params

    # Keep track of how the numbers change over time so we can see when they stop
        changing
    history = []

    for iteration in range(max_iters):
        # Save the current state of our parameters
        current_params = np.array([alpha, beta, gamma_p, lambda_0, lambda_1])
        history.append(current_params)

        # Let the model guess the probabilities using our current parameters
        gamma_C, gamma_Z = belief_update(X, alpha, beta, gamma_p, lambda_0,
            lambda_1)

        # Force the model to pick the most likely state
        C_hat = np.argmax(gamma_C, axis=1)
        Z_hat = np.argmax(gamma_Z, axis=2)

        # Pretend those hard guesses are the true data, and learn the new
            parameters
        alpha_new, beta_new, gamma_new, lambda_0_new, lambda_1_new =
            learn_parameters_complete(C_hat, Z_hat, X)

        new_params = np.array([alpha_new, beta_new, gamma_new, lambda_0_new,
            lambda_1_new])

        # Did the numbers stop changing? If the biggest update is smaller than our
            tiny tolerance (tol), stop the loop.
        max_diff = np.max(np.abs(new_params - current_params))
        if max_diff < tol:
            print(f"Done! The parameters settled after {iteration + 1} loops.")
            # Save the final best numbers before we exit
            history.append(new_params)
            return new_params, np.array(history)

        # Otherwise, update the parameters and run the loop again
        alpha, beta, gamma_p, lambda_0, lambda_1 = new_params

    return new_params, np.array(history)
```

Applying EM to real data

```
# Setup
data_folder = 'proj_HMM'
file_names = [f for f in os.listdir(data_folder) if f.endswith('.csv')]

# Sort files by number
file_names.sort(key=lambda f: int(''.join(filter(str.isdigit, f))))

# List to store parameters for each experiment
results = []

# Initial guess to help the algorithm start correctly
init_guess = (0.7, 0.4, 0.4, 1.5, 4.0)

# Loop through every data file
for file_name in file_names:
    file_path = os.path.join(data_folder, file_name)
    df = pd.read_csv(file_path, sep=None, engine='python')

    # Get the X matrix and drop 't' column if it exists
    if 't' in df.columns:
        X_real = df.drop(columns=['t']).values
    else:
        X_real = df.values

    # Run the Hard-EM algorithm
    # We use 30 iterations because it usually finishes fast
    final_params, _ = hard_em(X_real, init_guess, max_iters=30, tol=1e-4)

    # Save the final values for this test
    results.append({
        'Experiment': file_name,
        'alpha': final_params[0],
        'beta': final_params[1],
        'gamma': final_params[2],
        'lambda_0': final_params[3],
        'lambda_1': final_params[4]
    })

# Show the results in a table
results_df = pd.DataFrame(results)
results_df.set_index('Experiment', inplace=True)

print("Estimated Parameters for Real Experiments")
# Round numbers to 3 decimals
display(results_df.round(3))
```